

XX 電子

(EVCS_充電槍與模塊最佳化配置案_2026)
5-1-1. Technology Concept Design
Report

Author: Sonny Lee

Version: 1.0.0

Revision History

Date	Revision	Description	Author
2026/1/26	0.0.1	Initial Concept design document	Sonny Lee
2026/3/3	0.0.2	Delivery 3.Data structure 4.System architecture	Sonny Lee
2026/3/10	0.0.3	Updated sections 3 and 4	Sonny Lee
2026/3/13	0.0.4	Separated the architecture and flow chart, refactored the architecture design, and adjusted the order of the sections.	Sonny Lee
2026/3/16	0.0.5	Added section 4. Core classes	Sonny Lee
2026/3/17	0.0.6	Update section 2.3 time-step flow	Sonny Lee
2026/3/19	0.0.7	1. 修正前設計無法照應到 relay 切換之需求 2. 各物件屬性修正更新，並區分 public / private 3. 補充來車 Handshake 的 Sequence Diagram 4. MCU 間借還電 API 列出詳細內容 5.所有寫死的數字改為可配置變數	Sonny Lee
2026/3/20	0.0.8	1.於 Vehicle 類別加入屬性 request_power、max_power 2.從 TimeStep 流程中移除 Reset Env.	Sonny Lee
2026/3/31	0.0.9	1.修正借還電 API 方法的順序 2.Relay 切換責任釐清 3.xxx_power 變數歸屬釐清 4.邏輯補充功率提升與降低的判斷	Sonny Lee
2026/4/2	0.0.10	1. 補充借電/還電各 function 中的 Relay 切換說明 2. 設計錯誤日誌格式	Sonny Lee
2026/4/7	0.0.11	修正 Review 0.0.10 建議	Sonny Lee
2026/4/8	0.0.12	1.修正 MCUControl 類別:借還電 API 方法 2.刪除 Relay 類別的 is_cross_mcu 方法	Sonny Lee

Document Review Record

Revision	Date	Reviewer	Notes
1.0.0	2026/4/9	Chilung Wang	Approve

目錄

1. 系統概述	5
1.1 專案目標	5
2. 系統架構.....	5
2.1 Time Step 驅動架構.....	5
2.2 架構總覽	5
2.3 Time Step 流程	7
3. 資料結構的目的	10
3.1 Relay Matrix (繼電器關係合法矩陣)	10
3.2 Module Assignment (模組使用狀態矩陣)	11
3.3 共通性原則.....	11
3.4 借電邏輯(找資源).....	12
3.4.1 邏輯目的	12
3.4.2 步驟說明	12
3.5 還電邏輯 (放資源).....	12
3.5.1 邏輯目的.....	12
3.5.2 步驟說明	12
3.6 來車邏輯(資源管理)	13
3.6.1 邏輯目的.....	13
3.6.2 主動來車步驟說明.....	13
3.6.3 來車衝突(被通知還電)步驟說明	13
3.7 EVCS 多 MCU 環形通訊索引設計.....	14
3.7.1 環型 MCU 定址	14
3.7.2 硬體對應 (CAN Bus).....	14
4. RelayEventLog 設計.....	錯誤! 尚未定義書籤。
4.1 設計決策.....	錯誤! 尚未定義書籤。
4.2 事件序列範例.....	錯誤! 尚未定義書籤。
5. 核心類別.....	錯誤! 尚未定義書籤。
5.1 類別概述.....	錯誤! 尚未定義書籤。
5.2 Simulation Modules 類別	錯誤! 尚未定義書籤。
5.2.1 Vehicle 類別	錯誤! 尚未定義書籤。
5.2.2 TrafficSimulator 類別	錯誤! 尚未定義書籤。
5.2.3 VehicleGenerator 類別.....	錯誤! 尚未定義書籤。
5.2.4 MCUControl 類別	錯誤! 尚未定義書籤。
5.2.5 ChargingStation 類別	錯誤! 尚未定義書籤。
5.2.6 RectifierBoard 類別	錯誤! 尚未定義書籤。
5.2.7 SMRGroup 類別	錯誤! 尚未定義書籤。
5.2.8 SMR 類別	錯誤! 尚未定義書籤。
5.2.9 Relay 類別	錯誤! 尚未定義書籤。

5.2.10 Output 類別	錯誤! 尚未定義書籤。
5.2.11 RelayEvent 類別	錯誤! 尚未定義書籤。
5.2.12 RelayEventLog 類別	錯誤! 尚未定義書籤。
6. 日誌格式設計	錯誤! 尚未定義書籤。
附件：	錯誤! 尚未定義書籤。

圖目錄

圖 1：架構圖	6
圖 2：Time Step 流程圖	9

引言

本技術概念設計報告旨在詳細說明 EVCS (Electric Vehicle Charging Station) 充電站功率管理系統的技術架構、核心算法、性能需求及實現方案。

1. 系統概述

1.1 專案目標

本專案之核心目標為：

本年度 (2026.01~11) 目標為基於 EVCS 演算法進行三面向的擴展與優化：

面向	目標說明
1. 完整模擬引擎	A. Timing Diagram 視覺化:提供 Timing Diagram 以視覺方式呈現 Relays 在多種情境下的變化 B. 測試完整度提升:納入車輛種類、來車 SOC、預定充電量、來車密集率等參數，建立貼近實際營運的模擬環境 C. 將演算法移植到 MCU 硬體環境 (C 語言實現)，並完成性能驗證
2. Relay 切換最小化	優化演算法以減少 Relay 切換次數，降低硬體損耗與維護成本
3. 來車建議充電樁	開發推薦機制，根據系統狀態引導車輛至最佳充電樁位，提升用戶滿意度與整體利用率

2. 系統架構

2.1 Time Step 驅動架構

設計原則

原則	說明
時間步進為核心	固定時間間隔執行，不是事件觸發
輪詢式處理	每個 step 檢查所有狀態變化
同步執行	Environment → Modules
狀態快照	每個 step 產生完整的系統狀態快照

2.2 架構總覽

整體採用分散式設計，核心架構分為兩層：Simulation Environment 與 Simulation Modules。核心設計原則為無集中式協調者，每個 MCU 皆獨立自治運作，所有業務邏輯均限定在本地視角內完成。

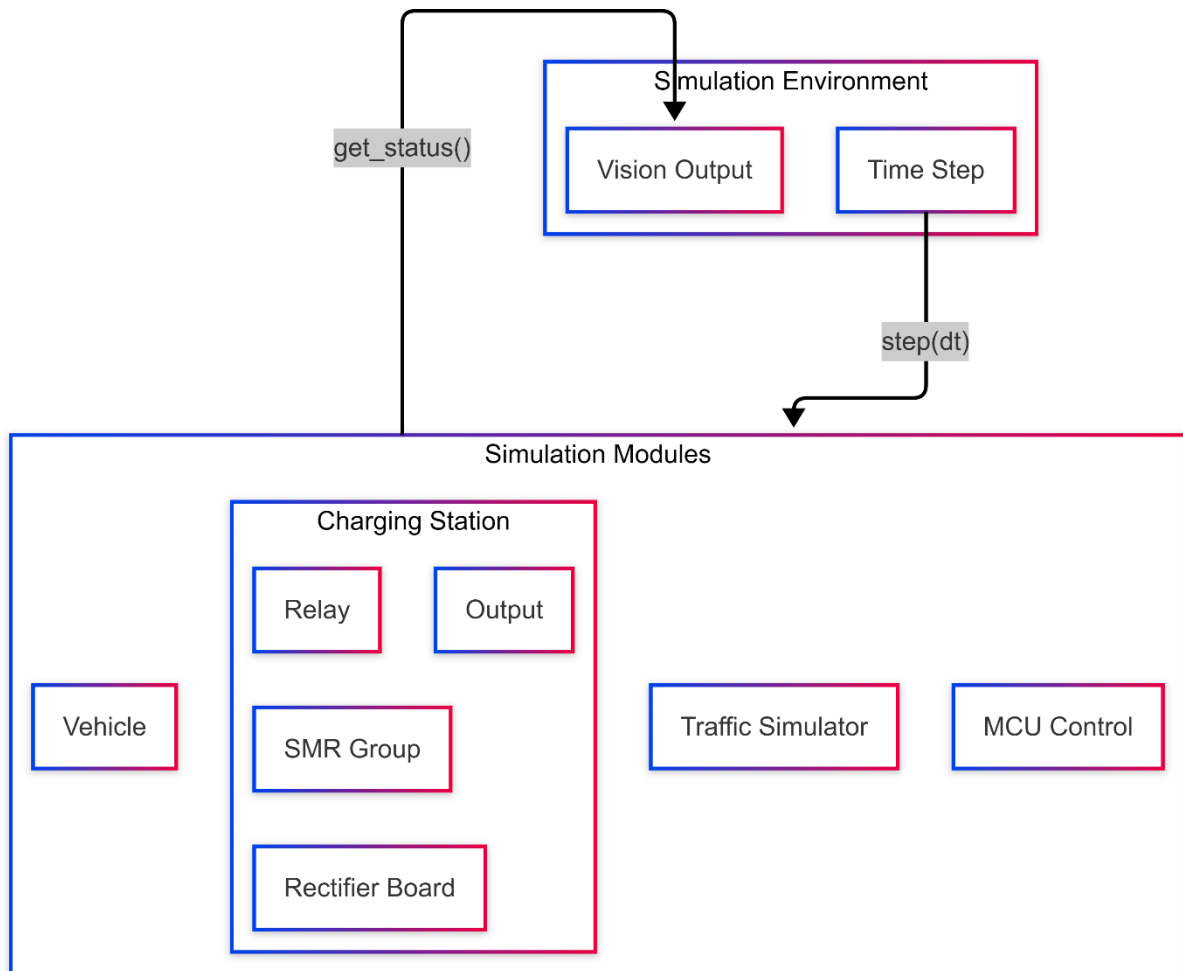


圖 1：架構圖

第一層：Simulation Environment（模擬環境）

系統的時間驅動者與輸出管理者，本身不執行任何業務邏輯，只負責協調整個模擬的節拍與結果收集。

Time Step

系統的心跳產生器。每個迴圈推進一個 dt ，對所有 Simulation Modules 發出 `step(dt)` 呼叫，驅動底層所有物件更新自身狀態。時間推進是唯一的驅動來源，沒有任何外部事件可以繞過這個機制直接觸發模組。

Vision Output

狀態收集與可視化輸出的管理者。透過 `get_status()` 從所有 Simulation Modules 收集各自的狀態快照，執行一致性驗證（參考附件說明），驗證通過後才輸出 Timing Diagram 與 Global View。驗證不通過則報錯，禁止輸出，確保可視化永遠呈現正確一致的狀態。為解決錯誤不透明的疑慮，報錯機制會包含：（1）明確標示發生不一致的 MCU 索引與邊界區塊；（2）記錄衝突的具體欄位與數值差異。另外，Relay 切換記錄的設計請參考 4.RelayEventLog 設計。

第二層：Simulation Modules（模擬模組）

所有模組平等並列，沒有主從關係，沒有集中式統籌者。每個 dt 收到 `step(dt)` 後各自執行 `update()`，再各自回報 `get_status()`。

Vehicle (車輛實例)

代表一輛正在充電或等待充電的電動車。持有該車輛的 SOC 與充電功率關係圖、初始 SOC、截止 SOC，在每個 dt 更新當前 SOC。車輛生成後直接與對應的 Charging Station (MCU) 互動，不經過任何中央管理者。

Traffic Simulator (來車模擬器)

車輛的來源產生器。根據設定的車輛密集度，決定是否在這個 dt 生成新的 Vehicle 實例，並將車輛路由到對應的充電槍。車輛一旦連接完成，Traffic Simulator 不再介入後續的充電互動，完全交由 Vehicle 與 MCU Control 自行處理。

MCU Control (MCU 控制器)

MCU 的業務邏輯核心，與其他 Module 地位平等，不是獨立的控制層。在每個 dt 的 update() 中自行判斷是否需要執行借電、還電、功率重新分配等決策，不等待外部指令。透過 Ring Topology 僅與左右相鄰 MCU 協商邊界，協商完成後回到 Local 自行處理。未來此模組的邏輯將移植為嵌入式 C 程式碼部署到真實 MCU 硬體。

Charging Station (充電站容器)

充電站實體設備的數位替身，同時作為充電站殼 (Shell) 的模擬容器。內部由四個子物件組成：

- **Rectifier Board (整流板)** 對應實體的 REC BD，是整個充電站硬體的頂層容器，同時作為 SMR Group 的主板。每塊 Rectifier Board 對應一組 MCU。
- **SMR Group (Switching Mode Rectifier 群組)** 整流模組群組，對應多顆 25kW 整流模組。MCU Control 決策後下達 Relay 切換指令，以決定 SMR Group 之間的串聯或斷開，進而調整輸出功率。
- **Relay (繼電器)** 直流繼電器的模擬物件，共有兩種類型：(1) 控制功率流向 Output 的通路開關；(2) 負責 SMR Group 之間的斷開與閉合。繼電器的切換時序是 Timing Diagram 的主要呈現內容。
- **Output (充電輸出)** 充電槍輸出端的模擬物件，代表實際對車輛輸出的直流功率通道。Vehicle 實例透過 Output 與 Charging Station 建立充電關係。

2.3 Time Step 流程

名詞定義

- **Max Require Power**：車輛更新 SOC 後，查詢自身功率曲表得出，表達「我現在需要多少功率」。
- **Available Power**：MCU Control 評估自身功率餘裕後產出，表達「我現在最多能給你多少功率」，是決策結果的依據。
- **Present Power**：車輛目前實際用電量，取得 Available Power 與 Max Require Power 其中的最小值，由車輛主動向 MCU Control 提出。

流程總覽

整個流程分為**初始化與主迴圈**兩個階段。 $t(k+1) = t(k) + \Delta T$ 與 $t(k) < t_end$ 判斷位於**迴圈最前端**，先推進時間、先判斷是否繼續，再執行本輪的模擬內容。

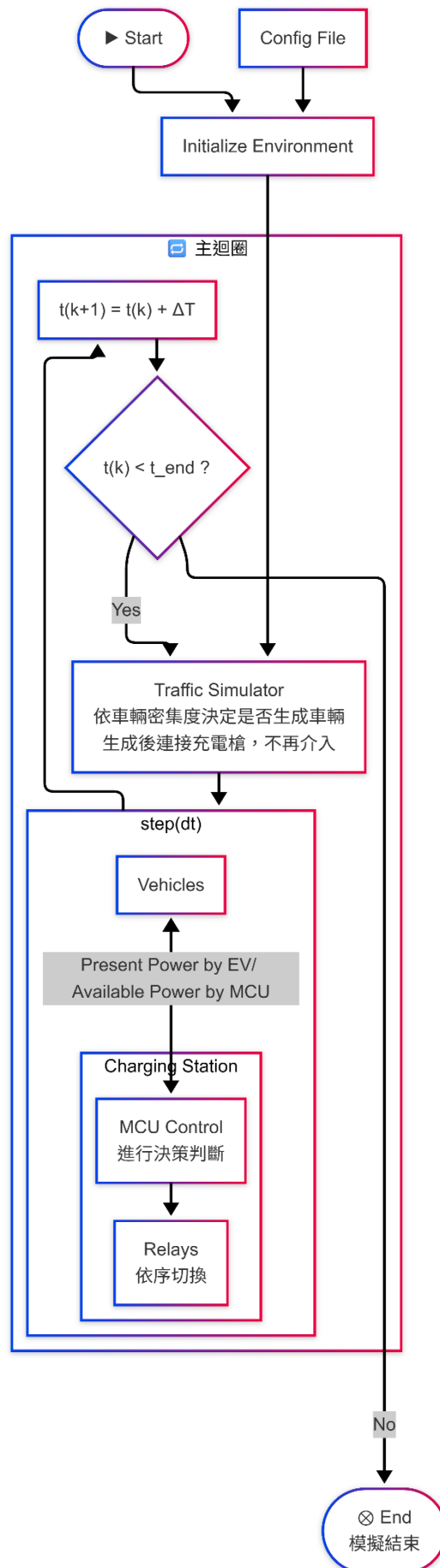


圖 2：Time Step 流程圖

初始化階段

系統啟動時，Start 與 Config File 共同輸入 Initialize Environment，完成環境建構與參數載入後，接著進入主迴圈

主迴圈

時間步進與終止判斷

$t(k+1) = t(k) + \Delta T$ 每輪迴圈的第一步先推進時間，k 加 1。

$t(k) < t_{end}$? 推進後立即判斷是否到達結束時間：

- **Yes:** 繼續往下執行本輪模擬內容
- **No:** 跳出主迴圈，往下到達 End

Traffic Simulator

判斷為 Yes 後進入 Traffic Simulator。依設定的車輛密集度，決定本 dt 是否生成新的 Vehicle 實例，若生成則連接至對應充電槍。連接完成後 Traffic Simulator 不再介入該產生的 Vehicle，後續完全由該 Vehicle 與系統其他模組互動完成。

step(dt)

這是每個時間步的核心，包含兩個平等並列的物件，以及它們之間的連鎖行為

設計原則

物件之間的互動屬於內部引發的連鎖行為。Vehicle 更新自身狀態後，MCU Control 會得到功率變化並做出對應決策，是各物件在 step(dt) 被觸發後，由自身邏輯引發的連鎖反應；Vehicles 與 MCU Control 都是連鎖行為的主體。兩者在每個 step(dt) 中各自被觸發，並透過共享的功率狀態資訊（Present Power / Max Require Power / Available Power）產生互動，這條資訊流是雙方內部邏輯引發的，而非外部安排的呼叫順序。

Vehicles

收到 step(dt) 觸發後，Vehicle 依序執行三個行為：

- 1、根據目前充電功率積分，更新 SOC 將上一個 dt 內實際獲得的充電功率乘以時間步長（ ΔT ），計算出本輪充入的電量，累加到當前 SOC。這是 Vehicle 狀態推進的起點。
- 2、根據新的 SOC，更新功率需求（Max Require Power）SOC 更新後，Vehicle 依據自身的充電曲線重新評估當前所需的充電功率，產出新的 Max Require Power。
- 3、先運算得出 Present Power 之後，對 MCU 送出新的 Present Power 訊息 Vehicle 完成狀態更新後，新的 Present Power 在本輪主動通知 MCU Control。

Charging Station

同樣收到 step(dt) 觸發，內部依序執行：

MCU Control（進行決策判斷）MCU Control 是 Charging Station 唯一具有行為的物件，也是所有決策的唯一來源。它由車輛通知的 Present Power，對照自身的 Available Power，執行借還電策略，判斷實際分配給這個 Vehicle 的功率數值。硬體的狀態（如各組功率數值）由 MCU Control 主動讀取與更新，硬體本身只提供狀態，不發起任何行為。

Relays（依序切換）依照 MCU Control 的指令執行繼電器切換。Relay 的切換結果更新硬體狀態。

End 模擬結束

當 $t(k) \geq t_{end}$ ，條件判斷輸出 No，流程跳出主迴圈，模擬正式結束。所有 Vehicle Instance、RectifierBoard、MCU 的最終狀態交由 Vision Output 收集，產出最終的 Global View 與 Timing Diagram。

3. 資料結構的目的

本系統使用兩張核心矩陣表（參考下方說明），作為管理充電站電力資源的「共同語言」。兩張表搭配使用，讓每個 MCU 都能清楚知道：哪些硬體開關（Relay）是合法的、哪些充電模組（Group）正在被誰使用。

3.1 Relay Matrix（繼電器關係合法矩陣）

這張表的核心目的是：定義哪些硬體連線是「物理上允許的」。Relay Matrix 告訴系統，哪兩個節點之間有實體電路連接，哪些組合根本不存在配線，永遠不可能接通（標記為 -1）。其中 MCU INDEX = 1 為本體，INDEX = 0, 2 分別代表左邊與右邊鄰居 MCU。

MCU		0				1				2				0		1		2	
	Group/Output	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	0	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0/1	-1	-1	-1	-1	-1
	1	0/1	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
	2	-1	0/1	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
	3	-1	-1	0/1	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	0/1	-1	-1	-1	-1
1	4	-1	-1	-1	0/1	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	0/1	-1	-1	-1
	5	-1	-1	-1	-1	0/1	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
	6	-1	-1	-1	-1	-1	0/1	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
	7	-1	-1	-1	-1	-1	-1	0/1	-1	0/1	-1	-1	-1	-1	-1	-1	0/1	-1	-1
2	8	-1	-1	-1	-1	-1	-1	-1	0/1	-1	0/1	-1	-1	-1	-1	-1	-1	0/1	-1
	9	-1	-1	-1	-1	-1	-1	-1	-1	0/1	-1	0/1	-1	-1	-1	-1	-1	-1	-1
	10	-1	-1	-1	-1	-1	-1	-1	-1	-1	0/1	-1	0/1	-1	-1	-1	-1	-1	-1
	11	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0/1	-1	-1	-1	-1	-1	-1	0/1
0	12	0/1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
	13	-1	-1	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
1	14	-1	-1	-1	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
	15	-1	-1	-1	-1	-1	-1	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
2	16	-1	-1	-1	-1	-1	-1	-1	-1	0/1	-1	-1	-1	-1	-1	-1	-1	-1	-1
	17	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	0/1	-1	-1	-1	-1	-1	-1

矩陣的橫軸與縱軸，分別代表系統中所有的充電模組 Group（INDEX 0-11）和充電槍

Output (INDEX 12-17)

- 值為 0：繼電器目前開啟（斷路）
- 值為 1：繼電器目前閉合（通路）
- 值為 -1：此兩節點之間無實體配線，永遠不合法

有了這張表，系統就不需要逐一查詢硬體配置，只需查表即可判斷是否「物理可行」？

3.2 Module Assignment (模組使用狀態矩陣)

這張表的核心目的是：記錄當下每個充電模組正在被哪個充電槍（Output）使用。其中 MCU INDEX = 1 為本體，INDEX = 0, 2 分別代表左邊與右邊鄰居 MCU。

- 值為 0：此模組目前空閒，未被任何充電槍使用
- 值為 1：此模組目前使用中
- 值為 -1：此模組無法分配給 Output
- $Gx \text{ 的 } \text{INDEX MOD } 4 = \text{該 Group 在其 MCU 內的位置 } (0,1,2,3)$
- $Gx \text{ INDEX } / 4 = \text{該 Group 屬於哪個 MCU } (0=M1, 1=M2, 2=M3)$

透過掃描這張表，系統可以迅速完成四種查詢：

- 1、可用性檢查：掃描某一 Group 行，全為 0 時，表示此 Group 空閒可用
- 2、模組歸屬：掃描某一 Group 行，找到值為 1 的那列，即知是哪個 Output 在使用中
- 3、路徑追蹤：掃描某一 Output 列，找出所有 1 即為分配給該 Output 之充電模組

MCU	Group/ Output	0				1				2			
		0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	0	0	0	0	0	0	0	-1	-1	-1	-1
	1	0	0	0	0	0	0	0	0	-1	-1	-1	-1
1	2	0	0	0	1	1	1	1	1	0	0	0	0
	3	0	0	0	0	0	0	0	0	0	0	0	0
2	4	-1	-1	-1	-1	0	0	0	0	0	0	0	0
	5	-1	-1	-1	-1	0	0	0	0	0	0	0	0

- 4、衝突檢測：掃描某一 Group 行，出現多個 1 時，表示發生衝突需要處理

3.3 共通性原則

借電、還電、來車三種情境，雖然觸發原因不同，但在「如何操作 Relay Matrix 與 Module Assignment 表」這件事上，都遵循同一套共通性原則：

共通原則	說明
先找錨點，再擴展	任何操作都從「找到充電槍的合法起始 Group（錨點）」開始，再往兩側展開
本地優先，外部次之	操作順序上，本地 MCU 的資源先動用，外部借入的資源後動用
連續區間，不可跳躍	所有使用中的 Group 必須形成連續的 [MIN, MAX] 區間，不允許中間有空洞

3.4 借電邏輯(找資源)

借電，是指某個充電槍（Output）需要更多電力時，向鄰近的空間充電模組（Group）申請使用權的過程。

借電條件（需要更多模組）：

Present Power = Available Power

且持續 N 個 step（例如 3 個 step）→ 借入一個額外 SMR Group

3.4.1 邏輯目的

在不違反物理配線（Relay Matrix）且不與現有使用者衝突（Module Assignment）的前提下，盡可能為充電槍取得足夠的電力資源。

3.4.2 步驟說明

- 1、【初始化：找錨點】用 Relay Matrix 查詢 Output_x 對應的合法 Group，找出起點 Gx（即「錨點」）。
- 2、【確定初始區間】從錨點出發，判斷它在 MCU 內的位置，確定最小範圍 MIN 與 MAX（錨點的相鄰兩個 Group）。
- 3、【擴展迴圈：優先本地】每次擴展都先確認下一個要納入的 Group 是否屬於本地 MCU，是的話優先往該側擴展（MIN-1 或 MAX+1）。
- 4、【跨 MCU 借用】當本地資源不足時，才往外（跨 MCU）繼續擴展 MAX，實現跨 MCU 借電。

(可參考附件 Pseudocode)

共通原則：整個過程中，MIN 到 MAX 形成一段「連續不間斷」的 Group 區間。借電就是不斷把這個區間往外推的過程。

3.5 還電邏輯(放資源)

還電，是指充電槍用電需求降低時，主動釋放它不再需要的充電模組，讓其他充電槍或充電站可以使用

還電條件（模組使用率不足）：

Available Power - Present Power > = 1 個 SMR Group

且持續 N 個 step → 還出一個 SMR Group

3.5.1 邏輯目的

在滿足自身電力需求的前提下釋放多餘資源，且優先歸還「跨 MCU 借來的」資源，讓本地資源最後再動。

3.5.2 步驟說明

還電的過程是「從外往內收縮」：把 [MIN, MAX] 區間從兩端向內縮減，直到剩下最小必要範圍為止。每次縮減時遵循以下順序：

- 1、若 MIN 和 MAX 都屬於外部（非本地）MCU → 從 MIN 端開始歸還，MIN 往右移（由外到內）
 - 2、若只有 MIN 屬於外部 MCU → 先歸還 MIN 端，MIN 往右移
 - 3、若只有 MAX 屬於外部 MCU → 先歸還 MAX 端，MAX 往左移
 - 4、若 MIN 和 MAX 都是本地資源：若 MIN 是錨點（最核心），則從 MAX 端歸還；否則從 MIN 端歸還
- (可參考附件 Pseudocode)
- 共通原則：還電是「先還外部的，最後才動本地的」。錨點是最後一個被碰的本地核心資源

3.6 來車邏輯(資源管理)

來車，是指新的電動車抵達某個充電槍（Output），系統需要替它分配充電模組（Group），並偵測是否與現有借電者發生資源衝突

3.6.1 邏輯目的

確保新到的車輛能取得至少二個合法的充電模組，同時若該模組已被其他充電槍借走，則主動通知對方歸還，保護來車的基本充電需求

3.6.2 主動來車步驟說明

- 1、【初始化：找錨點】用 Relay Matrix 查詢 Output_x 對應的合法 Group，找出起點 Gx（即「錨點」）。
- 2、【確定初始區間】從錨點出發，判斷它在 MCU 內的位置，確定最小範圍 Gmin 與 Gmax。
- 3、【衝突偵測：掃描 Module Assignment 表】用 Module Assignment 表掃描 Gmin 到 Gmax 範圍內的 Group，若某個 Group 已有別的 Output 在用（值為 1），即代表「衝突」發生。
- 4、【通知還電：請借電方主動釋放】一旦偵測到衝突，系統向佔用該 Group 的充電槍（借電方）發出「釋放資源」的通知，由借電方執行還電流程（參考 3.6.3）。

3.6.3 來車衝突(被通知還電)步驟說明

- 1、【收到通知】被通知需釋放某個 Group（設為 Gx）。
 - 2、【找自己的錨點】用 Relay Matrix 找到自己充電槍的錨點位置。
 - 3、【判斷歸還】
 - 若錨點 < Gx，代表 Gx 在右邊，設 MAX = Gx-1，將所有大於 MAX 的 Group 歸還；
 - 若錨點 > Gx，代表 Gx 在左邊，設 MIN = Gx+1，將所有小於 MIN 的 Group 歸還。
 - 4、【執行 Relay 切換】先斷開 (Open) 所有 Relay 再依序閉合 (Close) (排除歸還的 Group)
- 共通原則：用 Gx INDEX 與錨點比較數字大小即可判斷出需歸還的 Groups

3.7 EVCS 多 MCU 環形通訊索引設計

在多 MCU 的環形拓撲架構中，每個 MCU 模組在不需要指標（pointer）的情況下，透過 index 運算自動找到「前一個」與「下一個」模組

3.7.1 環型 MCU 定址

前一個 MCU index = (自身 MCU index - 1) mod N

下一個 MCU index = (自身 MCU index + 1) mod N

其中 N = 系統中的 MCU 總數

特性：

- 無論 N 為多少，公式完全相同
- index = 0 的前一個為 N-1（環形繞回正確處理）
- 每個 MCU 只需知道自己的 index，不需要全域參數

3.7.2 硬體對應 (CAN Bus)

- 硬體上各板子透過 CAN Bus 連接，需要指定 CAN Bus ID
- 每個節點知道自己的 ID，用上述公式算出前後節點的 ID，即可建立通訊